

Collision Risk Analysis and Prediction in Satellite Orbits

Submitted by:

Srishti Singh – 220905526

Pranay Goel – 220905524

Under the guidance of:

Dr. Manjunath K.N.

Department of Computer Science Engineering

Minor Specialization:

Big Data Analytics



Department of Computer Science and Engineering

October 2025

Abstract

This project implements a scalable pipeline for collision risk prediction among Earth-orbiting objects by integrating Two-Line Element (TLE) datasets with distributed storage and computation frameworks — principally Hadoop and Apache Spark. The pipeline automates acquisition, ingestion, and preprocessing of TLEs, executes large-scale orbital propagation using the SGP4 model, and performs pairwise conjunction analysis using Spark-based distance computations and windowed aggregation.

The system stores intermediate and final products in HDFS and exposes analytical results through Streamlit for query and reporting. Cluster-level optimizations such as broadcast variables, adaptive shuffle partitions, and columnar Parquet storage are employed to achieve scalable performance. The deliverable is an auditable, reproducible framework capable of daily runs for near-term collision detection, accompanied by documented cluster installation, scripts, and troubleshooting notes. An interactive Streamlit-based dashboard was also developed to visualize conjunction data retrieved dynamically from HDFS outputs.

Table of Contents

1. Introduction.....	1
2. Literature Review.....	3
3. Methodology.....	5
4. Two-Line Element Set (TLE): Format Overview.....	7
4.1 Structure of a TLE.....	7
4.2 Line 1 Fields.....	7
4.3 Line 2 Fields.....	8
4.4 Example: Interpretation.....	8
5. Implementation and Results.....	9
5.1 Cluster Implementation and Setup.....	9
5.2 Deployment Mode Comparison: YARN vs. Spark Standalone.....	10
5.3 Data Ingestion and Preprocessing.....	10
5.4 Propagation Stage.....	10
5.5 Conjunction Detection and Aggregation.....	11
5.8 Execution and Testing.....	12
5.9 Implementation Challenges and Resolutions.....	13
5.10 Results and Empirical Validation.....	13
5.10.1 Cluster Configuration and Verification.....	13
5.10.2 Pipeline Execution and Logs.....	16
5.10.3 Analytical Dashboard.....	18
5.10.4 Performance Validation.....	22
5.10.5 Empirical Insights.....	22
6. Conclusion and Future Work.....	23
6.1 Summary of Outcomes.....	23
7. References.....	24
8. Appendix.....	25

1. Introduction

1.1 Background and Motivation

The environment of Earth's orbit, particularly Low Earth Orbit (LEO), is increasingly congested due to the rapid deployment of mega-constellations and growing space debris. This congestion has significantly heightened the risk of orbital collisions, which threaten valuable assets and generate additional debris, accelerating the problem. Proactive collision risk prediction and space traffic management are now essential for the long-term sustainability of space operations.

1.2 Problem Statement

The accumulation of space debris in Earth's orbit has significantly heightened the risk of **in-orbit collisions**. Traditional satellite tracking and collision prediction methods are often constrained by computational limitations and lack scalability to manage the exponentially growing volume of orbital data. With thousands of active satellites and millions of debris fragments currently monitored, the computational demand for **pairwise conjunction analysis** becomes immense.

Existing solutions rely heavily on sequential processing, leading to significant inefficiencies when performing **real-time or near real-time risk assessments**. This limitation hampers the timely identification and mitigation of potential satellite collisions, especially as the number of objects in orbit continues to rise. Additionally, major organizations such as **NORAD** and **USSPACECOM** employ proprietary big data frameworks and closed pipelines for orbital tracking. While effective, these solutions often lack transparency, reproducibility, and accessibility for the broader scientific and commercial community.

Therefore, there is a critical need for a scalable, efficient, and accurate system capable of ingesting, processing, and analysing **large-scale orbital datasets** using open-source distributed computing technologies such as **Apache Hadoop** and **Apache Spark**. Such a system must not only handle historical datasets but also be capable of streaming and analysing real-time telemetry data, ensuring continuous updates to collision risk assessments.

By integrating scalable big data frameworks with satellite data ingestion and predictive analytics, this solution can democratize access to space situational awareness and improve the global response to orbital threats. This project aims to fulfil this need by developing a big data-driven pipeline utilizing Hadoop and Apache Spark to automate the ingestion of **TLE data**, propagate satellite orbits using the **SGP4 algorithm**, and identify potential **close-approach events**. The system will be designed to support both **real-time** and batch-mode risk analysis and can be further enhanced with visualization tools to improve situational awareness.

1.3 Objectives of the Project

The primary goal is to develop a Big Data-driven pipeline using Hadoop and Apache Spark to automate satellite collision risk prediction. Key objectives include:

- **Data Ingestion and Storage:** To automate the process of acquiring large-scale Two-Line Element (TLE) datasets from public sources (e.g., CelesTrak) and store them in the **Hadoop Distributed File System (HDFS)**.
- **Preprocessing:** To implement data cleaning and transformation using Apache Spark to convert raw TLE data into a structured **Parquet** format for efficient distributed processing.
- **Orbital Propagation:** To implement the **SGP4 orbital propagation model** at scale to simulate future 3D positions and velocities of all space objects over a configurable time window.
- **Conjunction Detection:** To perform large-scale pairwise distance calculations and identify close-approach events (conjunctions) by applying predefined distance and relative velocity thresholds.
- **Scalability Demonstration:** To evaluate the time and resource efficiency of the system compared to traditional methods and demonstrate the performance benefits of a Spark Standalone cluster.

1.4 Scope and Relevance

The project's scope encompasses the entire Big Data pipeline, from automated data downloading to the final conjunction result dashboard. The pipeline is designed for both live and offline analysis. The system's relevance lies in its ability to contribute to proactive space traffic management and debris mitigation by providing a scalable, efficient, and transparent method for risk assessment using open-source technologies. Additionally, a Streamlit-based dashboard has been implemented to provide an interactive visualization interface. Once the computational pipeline completes execution, the conjunction results stored in HDFS are accessed by the dashboard via a symbolic link. This design enables real-time display and analysis of satellite conjunction data without manual intervention or data movement.

2. Literature Review

The literature underpinning this project spans **orbital propagation methodologies**, **operational conjunction assessment systems**, and the **application of big data technologies** to space surveillance and collision risk prediction.

2.1 Orbital Propagation and SGP4

The **Simplified General Perturbations Model 4 (SGP4)** is a widely adopted analytical orbit-propagation model that ingests **Two-Line Element (TLE)** sets to compute approximate satellite positions.

Previous studies — notably **Vallado and Crawford (2006, 2008)** — examined SGP4 implementation practices and confirmed its suitability for operational conjunction assessment, balancing computational efficiency with propagation accuracy. In collision risk prediction, **SGP4** remains foundational due to its proven reliability and adaptability to large-scale datasets used in modern space situational awareness applications.

2.2 Operational Conjunction Assessment Systems

Government and space agency systems such as **NASA's CARA (Conjunction Assessment Risk Analysis)** and **ESA's DRAMA (Debris Risk Assessment and Mitigation Analysis)** represent established operational frameworks for collision risk evaluation. These systems employ probabilistic and deterministic models to predict potential conjunctions between active satellites and debris. However, as the number of tracked objects increases exponentially, these traditional infrastructures face challenges in scalability and near-real-time performance. Historical analyses — such as **Kelso's study of the Iridium 33–Cosmos 2251 collision** — further emphasize the growing importance of proactive and automated collision monitoring systems.

2.3 Big Data Approaches for Space Surveillance

Recent research demonstrates the efficacy of **Big Data frameworks**, such as **Apache Hadoop** and **Apache Spark**, for managing and processing large-scale space datasets. For instance, **Kim et al. (2017)** applied Spark for near-real-time space object analysis, validating the feasibility of distributed processing for telemetry and orbital computations. Building on these advances, this project leverages Big Data technologies to integrate **SGP4-based orbital propagation** with **Spark-optimized pairwise computations**, enabling scalable and efficient conjunction detection.

2.4 Research Gap

While established systems offer accurate assessments, they often suffer from limited scalability, proprietary restrictions, or lack of reproducibility. A publicly documented, **open-source, and benchmarked pipeline** that handles the **end-to-end process** — from TLE ingestion and SGP4 propagation to conjunction detection — remains absent. This project directly addresses this gap by developing a **reproducible Big Data pipeline** optimized for small, dedicated clusters. It also benchmarks performance improvements by comparing **Spark Standalone mode** against **YARN**, demonstrating superior efficiency for compact, resource-constrained deployments.

3. Methodology

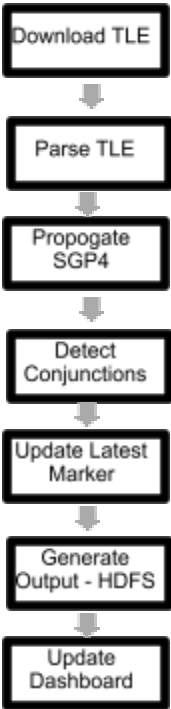
The methodology centers on a multi-stage Big Data pipeline deployed on a dedicated Hadoop/Spark cluster.

3.1 Architecture and Workflow

The system utilizes a 4-node cluster (1 Master, 3 Workers). The core processing follows a linear, staged workflow orchestrated by Python scripts:

Stage	Script	Objective	Output Storage
0	download_tle.py	Fetch latest TLEs from CelesTrak.	HDFS: /user/bda/tle/raw/<DATE> (Text files)
1	parse_tle.py	Convert raw TLE files into structured, valid records.	HDFS: /user/bda/tle/processed/<DATE> (Parquet)
2	propagate_sgp4.py	Use SGP4 to compute positions (x, y, z) and velocities (vx, vy, vz) over time.	HDFS: /user/bda/tle/positions/<DATE> (Parquet)
3	check_conjunctions.py	Detect close approaches (within 5 km) between satellite pairs.	HDFS: /user/bda/tle/conjunctions/<DATE> (Parquet)

Pipeline Flow:



3.2 Tools, Frameworks, and Datasets

- **Hadoop (v3.3):** Used for the **HDFS NameNode** and **DataNodes** to provide reliable, distributed storage for the TLE data and intermediate Parquet files.
- **Spark (v3.5.x):** The core engine for distributed processing, managing the large-scale SGP4 propagation and the $O(N^2)$ pairwise conjunction detection.
- **SGP4 Model:** Integrated via Python libraries to calculate future orbital parameters.
- **Datasets:**
 - **TLE Dataset:** Orbital parameters for active satellites and debris (e.g., *active_gp.tle*, *cosmos1408_debris.tle*).
 - **SATCAT Dataset:** A static reference CSV file mapping **NORAD ID** to satellite owner/country, used for enrichment.

3.3 System Design and Deployment

The project utilized a small-scale, dedicated cluster configuration:

- **Hardware Setup:** 1 Master Node (28 cores, 32 GB RAM) and 3 Worker Nodes (3 × 28 cores, 3 × 32 GB RAM), totaling 84 cores and 96 GB RAM for workers.
- **HDFS Structure:** The daily processing creates dated partitions, ensuring data integrity and enabling time-series analysis.
- **Deployment Mode:** The final production mode selected was **Spark Standalone Cluster** (*spark://master: 7077*) due to its superior performance over YARN for small, dedicated clusters.

3.3.1 Visualization Layer Integration

The visualization layer was implemented using Streamlit to provide an interactive dashboard for analytical review. The dashboard accesses the conjunction outputs stored in HDFS through a symbolic link, automatically loading the latest results after each pipeline execution. This component represents the end of the data flow.

3.4 Algorithm Details: Conjunction Detection

The conjunction detection stage involves a complex, optimized sequence of Spark transformations:

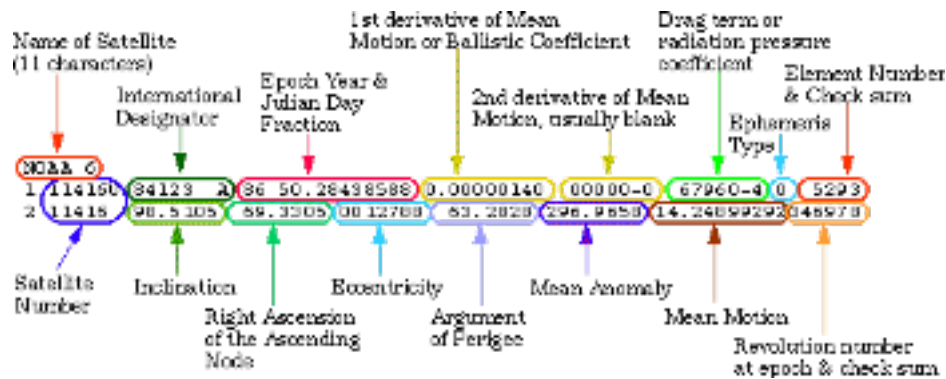
1. **Self-Join Optimization:** The positions dataset is joined with itself on the timestamp column, ensuring **satnum1 < satnum2** to avoid redundant checks and excludes debris-debris pairs to reduce computation by ~ 40%.
2. **Euclidean Distance and Filtering:** The 3D Euclidean distance is computed, and pairs are filtered where the distance is less than the **5.0 km threshold**.
3. **Relative Velocity:** Relative velocity is calculated, and pairs must exceed the **0.1 km/s threshold**.
4. **Time Clustering and Aggregation:** Conjunctions are grouped within a **60-minute time window** using Spark Window functions to compute the minimum distance and closest approach time per cluster.
5. **Enrichment:** The results are enriched using a **Broadcast Join** with the small SATCAT lookup table to add country/owner information.

4. Two-Line Element Set (TLE): Format Overview

Accurate orbit propagation depends on well-defined orbital parameters that describe the geometry and motion of an Earth-orbiting object. The **Two-Line Element Set (TLE)** is a standardized data format developed by NORAD for representing these parameters. It provides all essential elements required by propagation models such as **SGP4** and **SDP4**.

4.1 Structure of a TLE

A TLE consists of **two lines** of fixed-width text containing the satellite's orbital elements, optionally preceded by a name line (Line 0).



- **Line 0** – Object name (up to 24 characters).
- **Line 1** – Identifiers, epoch, and orbital decay information.
- **Line 2** – Orbital geometry and motion parameters.

4.2 Line 1 Fields

Field	Description
Line number	Always 1
Satellite catalog number	Unique NORAD ID (e.g., 25544 for ISS)
Security classification	U = Unclassified (public); S = Secret (not public)
International designator	Launch year + launch number + piece ID (e.g., 98067A → 1998, 67th launch, piece A)
Epoch	Reference time of orbital elements (YYDDD.DDDDDDDD → day of year)
1st time derivative of mean motion / 2	Orbital decay rate (rev /day ²)
2nd time derivative / 6	Usually, 0 for public TLEs
BSTAR drag term	Atmospheric drag coefficient used by SGP4
Ephemeris type	0 = SGP4/SDP4 (public), 1–5 = other models
Element set number	Incremented on each update
Checksum	Remainder of digit sum ÷ 10 (for error checking)

4.3 Line 2 Fields

Field	Description
Line number	Always 2
Satellite catalog number	Repeated for verification
Inclination (°)	Angle between orbital plane and Earth's equator
Right Ascension of Ascending Node (°)	Longitude where the satellite crosses the equator northward
Eccentricity	Shape of the orbit (decimal point omitted)
Argument of Perigee (°)	Orientation of the orbit's closest point to Earth
Mean Anomaly (°)	Satellite's position in orbit at epoch
Mean Motion (rev/day)	Orbits completed per day
Revolution Number at Epoch	Total orbits completed (last 5 digits only)
Checksum	As on Line 1

4.4 Example: Interpretation

From the ISS TLE example:

- The object is the **International Space Station (ZARYA)**, launched in 1998.
- Epoch = **20300.83097691**, meaning 2020 + 300.83 days → late October 2020 (UTC).
- Inclination = 51.64°, indicating a mid-latitude orbit.
- Mean motion = 15.49 rev/day → ~93 minutes per orbit.
- The **BSTAR term** indicates minor atmospheric drag effects typical of low-Earth orbits.

4.5 Importance in This Project

All satellite and debris trajectories analyzed in this project originate from TLE data obtained from **CelesTrak**. Each downloaded TLE file is timestamped and stored in **HDFS** under `/user/bda/tle/raw/<DATE>` for traceability. The project's Spark pipeline parses, validates, and propagates these TLEs using the **SGP4** model to compute state vectors (x, y, z, vx, vy, vz), which are then used for conjunction detection and risk evaluation.

5. Implementation and Results

This section details the setup, configuration, and execution of the distributed Hadoop–Spark environment, along with the performance evaluation and key optimizations applied during the development of the satellite collision risk prediction pipeline.

5.1 Cluster Implementation and Setup

The project was deployed on a **4-node Hadoop–Spark cluster** (1 master and 3 worker nodes). The installation and configuration were performed as follows:

- **Preparation:** Installed **Ubuntu 20.04/22.04**, **OpenJDK 11**, and created a dedicated user account **bda** on all nodes. Password less SSH and sudo configurations were set up to enable remote execution.
- **Network Configuration:** Each node was assigned a static hostname (master, worker1, worker2, worker3) with consistent entries in `/etc/hosts` to ensure seamless resolution across nodes.
- **SSH Setup:** Configured password less SSH from the master node to all workers using `ssh-keygen` and `ssh-copy-id`, enabling automatic execution of cluster control scripts.
- **Hadoop Installation:** Installed **Hadoop 3.3.6** under `/home/bda/hadoop`. Configured `core-site.xml`, `hdfs-site.xml` (replication factor = 3), and `yarn-site.xml` with the ResourceManager hosted on the master node. Environment variables (`JAVA_HOME`, `HADOOP_HOME`, `HADOOP_CONF_DIR`) were added to `.bashrc` and synchronized across all nodes.
- **Spark Installation:** Installed **Spark 3.5.7** under `/usr/local/spark` on each node, defined `SPARK_HOME`, and added Spark binaries to `PATH`. Configuration files were replicated using a distribution script.
- **Optional Components (Not Utilized) :** Hive was deployed with an embedded Derby metastore, and Hue was configured for exploratory testing; however, the final pipeline operated on the Spark standalone cluster without YARN integration.
- **Issue Resolution:**
 - Resolved **Hadoop daemon failures** by downgrading Java from version 21 to 11.
 - Fixed **clusterID mismatches** caused due to namenode reformatting.
 - Verified all HDFS daemons and Spark workers were operational before job execution.

5.2 Deployment Mode Comparison: YARN vs. Spark Standalone

The project benchmarked **YARN** and **Spark Standalone** modes to evaluate performance differences under identical workloads.

Metric	YARN	Spark Standalone	Improvement
Total Runtime	2.3 hours	13 minutes	10.6× faster
Shuffle Read Time	45 minutes	4 minutes	11.2× faster
Active Cores	56 (2 workers)	84 (3 workers)	50% more
Memory Utilization	54 GB (fragmented)	81 GB (full)	50% better

Analysis of YARN Slowdown:

- **Two-Tier Scheduling Overhead:** The ResourceManager → ApplicationMaster → Executor cycle adds latency in small clusters.
- **Resource Loss:** The ApplicationMaster process occupies one full worker node, reducing compute capacity by ~33%.
- **Container Allocation Latency:** YARN introduces delays between stages due to container request cycles.

Conclusion: Spark Standalone mode yielded markedly superior performance and resource utilization, making it the optimal choice for small, dedicated clusters.

5.3 Data Ingestion and Preprocessing

Raw TLE files are downloaded and stored in **HDFS** at /user/bda/tle/raw/<DATE> using download_tle.py.

The parse_tle.py Spark job validates and normalizes TLE pairs, extracts metadata (satnum, name, object_type, line1, line2), and writes structured Parquet files to /user/bda/tle/processed/<DATE>. This transformation ensures schema consistency and efficient downstream access.

5.4 Propagation Stage

The **propagate_sgp4.py** script reads processed Parquet data and performs **orbit propagation** using the **SGP4 algorithm** over a **7-day forecast window** at **10-minute intervals**. Each propagation task is parallelized by satnum and outputs datasets to /user/bda/tle/positions/<DATE>/, containing columns for position (x, y, z) and velocity (vx, vy, vz) in kilometers.

5.5 Conjunction Detection and Aggregation

The `check_conjunctions.py` job performs time-based self-joins on the propagated dataset to compute **3D separation distances** and **relative velocities** between unique satellite pairs. Filtering thresholds:

- **Distance** < 5 km
- **Relative velocity** > 0.1 km/s

The results are aggregated into 1-hour windows, with each conjunction event enriched by SATCAT owner metadata. The final conjunction results are stored in HDFS at `/user/bda/tle/conjunctions/<DATE>/` in Parquet format.

Additionally, they are dynamically linked to the Streamlit dashboard, allowing automated loading and visualization immediately after each execution cycle.

5.6 Dashboard Deployment and Visualization

To enhance accessibility and provide intuitive visualization of analytical outcomes, a **Streamlit-based dashboard** was developed and deployed as part of the project. The dashboard serves as the **interactive front-end** for the Big Data pipeline, enabling real-time monitoring, exploration, and analysis of results.

Key Features:

- **Real-Time Monitoring:** Displays job progress, execution time, CPU/memory utilization, and data throughput metrics by reading logs and Spark UI endpoints.
- **Interactive Visualizations:** Presents 2D and 3D plots of orbital trajectories, conjunction events, and minimum distance timelines using libraries such as *Plotly* and *Matplotlib*.
- **Data Exploration Interface:** Allows users to browse and filter datasets stored in HDFS, including TLE, propagated orbit, and conjunction outputs.
- **Query and Filtering Tools:** Supports user-defined filters for satellite IDs, time ranges, proximity thresholds, and velocity constraints.
- **Administrative Controls:** Provides utilities for triggering new pipeline executions, refreshing datasets, and exporting reports in CSV or PDF formats.
- **Deployment:** The dashboard runs as a **Streamlit web application** hosted on the master node and accessible across the cluster network (e.g., `http://master:8501`).

This visualization layer bridges the computational backend and end-user interaction, providing a comprehensive view of system performance and conjunction analysis outcomes.

5.7 Key Optimizations

Category	Technique	Benefit
Computation	Smart filtering (exclude debris–debris pairs)	Reduces join size by ~40%
Computation	Broadcast joins (SATCAT lookup)	Avoids costly network shuffles
Computation	Window partitioning (time-based clustering)	Enables efficient local aggregation
Storage	Parquet columnar format	Speeds up analytical reads and reduces I/O
Configuration	Tuned shuffle partitions (200–300)	Matches the 84-core cluster for balanced parallelism

5.8 Execution and Testing

Example Spark submissions during testing:

```
$SPARK_HOME/bin/spark-submit --master spark://master:7077 --executor-cores 28 --executor-memory 27G scripts/parse_tle.py
```

```
$SPARK_HOME/bin/spark-submit --master spark://master:7077 --executor-cores 28 --executor-memory 27G scripts/propagate_sgp4.py
```

```
$SPARK_HOME/bin/spark-submit --master spark://master:7077 --executor-cores 28 --executor-memory 27G scripts/check_conjunctions.py
```

Verification steps included:

- **HDFS I/O tests:** Confirmed read/write operations via `hdfs dfs -ls` and `-cat`.
- **Functional tests:** Ran Spark WordCount examples to validate resource distribution.
- **Monitoring:** Verified job execution through **Spark Web UI** (master:7077) and **YARN ResourceManager UI** (master:8088).
- **Logs:** Analyzed application logs in `$HADOOP_HOME/logs/` and `~/bda_proj/logs/` for performance metrics and error diagnosis.

The Streamlit dashboard was also verified to correctly reflect updated conjunction data following each pipeline run. Testing confirmed the validity of the symbolic link to the HDFS output directory and the successful refresh of visualization elements after data updates.

5.9 Implementation Challenges and Resolutions

Issue	Description	Resolution
Java incompatibility	Hadoop 3.3.6 failed under Java 21	Standardized on OpenJDK 11
SSH/key errors	Password less SSH misconfigurations	Reinitialized keys using ssh-keygen and ssh-copy-id
ClusterID mismatch	Dynamic IPs caused DataNode–NameNode mismatches	Synchronized master’s clusterID to all workers
Missing Hadoop JARs	Absent hadoop-streaming.jar in some nodes	Verified \$HADOOP_HOME consistency and re-synched binaries

5.10 Results and Empirical Validation

The complete pipeline was successfully deployed and executed across a four-node Hadoop–Spark cluster comprising one master and three worker nodes. Validation of both functionality and performance was achieved through system logs, Spark UI monitoring, and an interactive Streamlit-based analytical dashboard.

5.10.1 Cluster Configuration and Verification

Before execution, environment variables for Hadoop, Spark, and Hive were permanently exported via the *.bashrc* configuration file, as shown in **Figure 5.1**. This ensured consistent access to binaries and libraries across all nodes.



Figure 5.1: Environment variable setup in `.bashrc` for Java, Hadoop, Spark, and Hive configuration. (*bashrcfile.png*)

Cluster health was verified using the `spark://master:7077` web interface. The Spark Master dashboard (Figure 5.2) indicated that all three workers were active, each contributing 28 cores and 30 GiB of memory, totalling 84 cores and 90 GiB cluster memory.



Figure 5.2: Spark Master web UI showing active worker nodes and completed job executions. (sparkmaster.png)

Performance metrics during runtime, including CPU utilization, memory load, and disk I/O, were monitored through system resource visualizations for each worker node (Figures 5.3–5.5). These results confirm near-maximum CPU utilization (97–100%) across all nodes, with stable memory and network throughput, demonstrating optimal parallelization and balanced workload distribution.

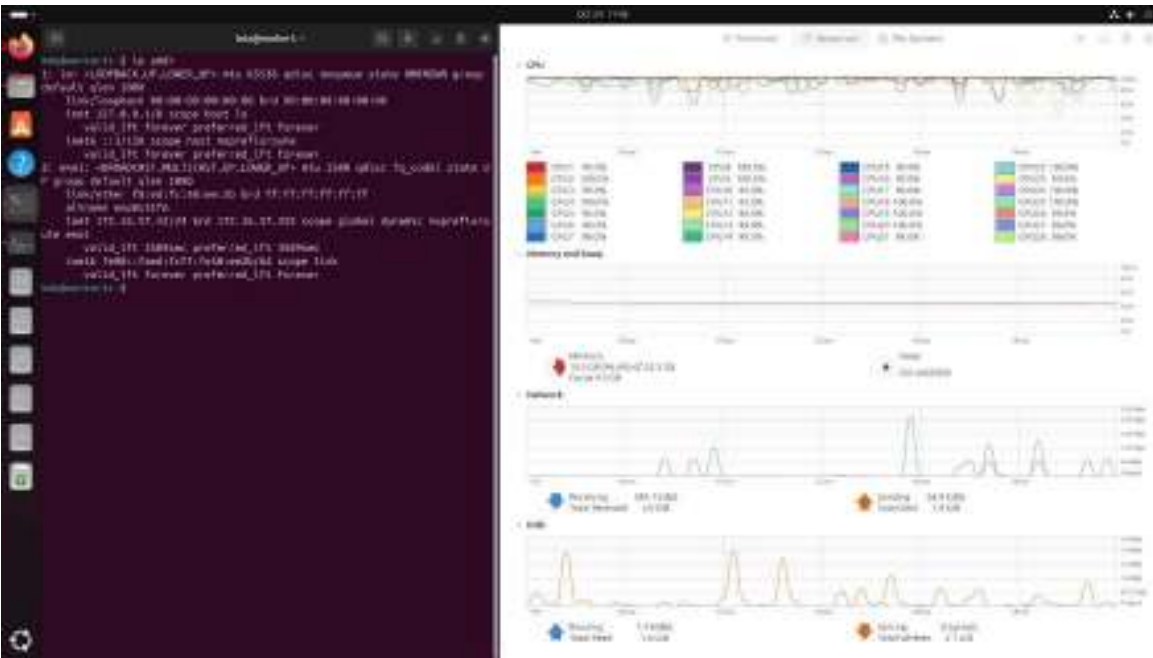


Figure 5.3: Worker 1 CPU and memory utilization during active Spark jobs. (worker1.png)

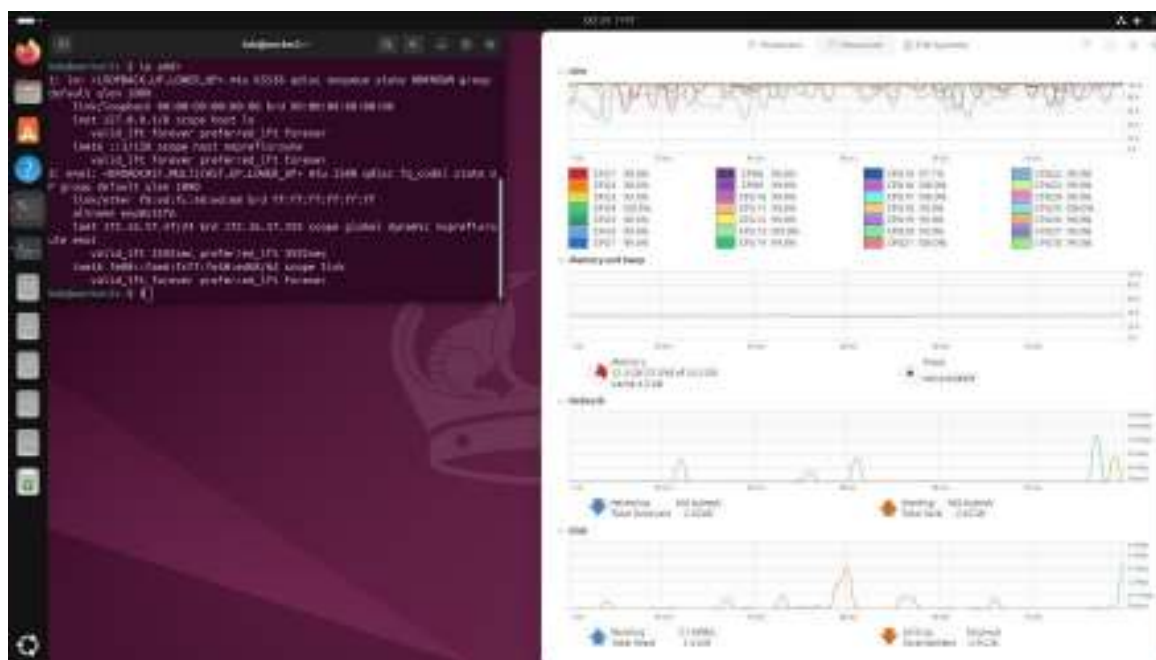


Figure 5.4: Worker 2 CPU and memory utilization showing consistent 98–100% parallel load.
(worker2.png)

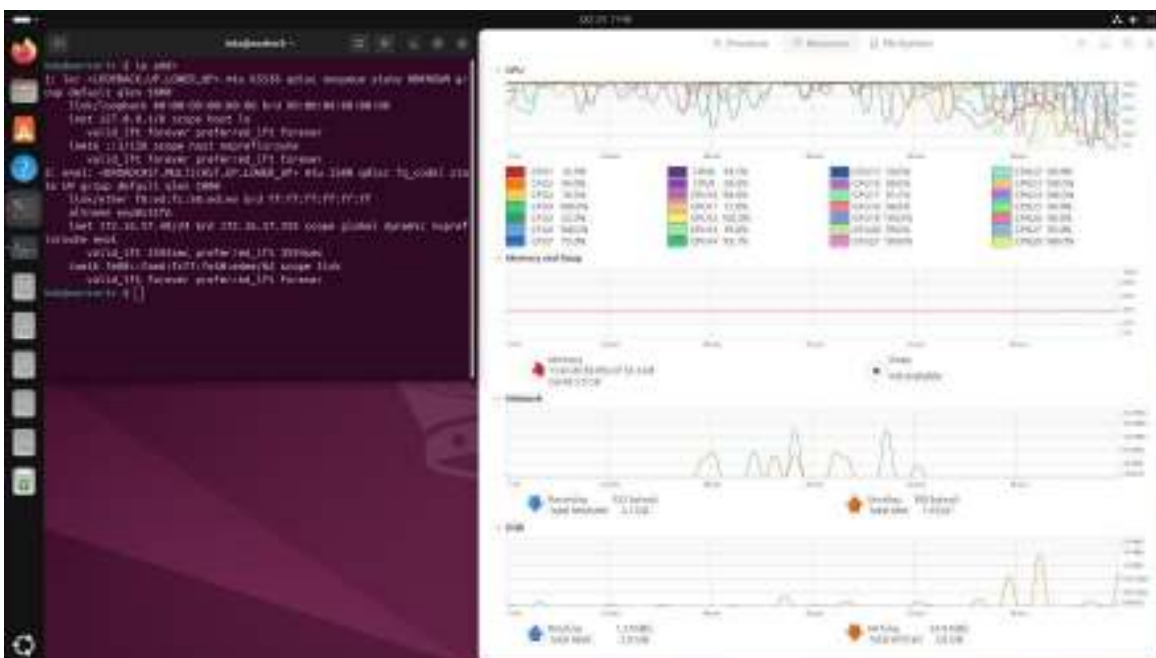
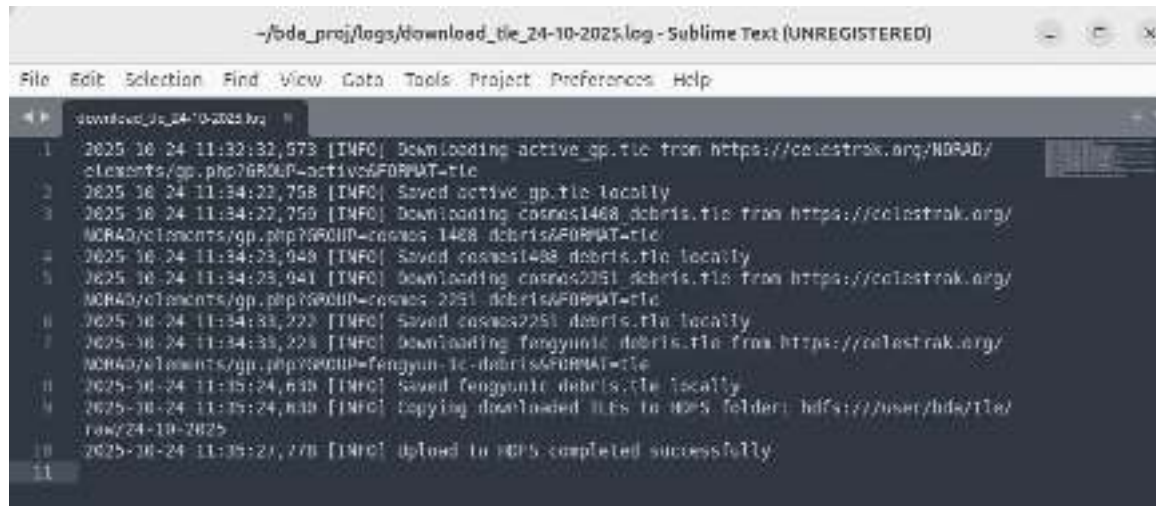


Figure 5.5: Worker 3 resource usage confirming uniform load balancing.
(worker3.png)

5.10.2 Pipeline Execution and Logs

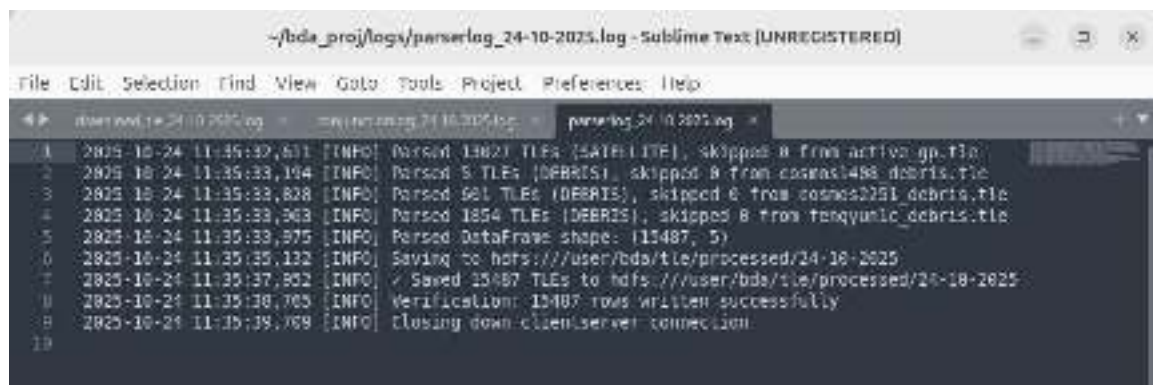
The pipeline execution followed sequential stages: TLE download, parsing, propagation, conjunction analysis, and output logging.

- **Download Stage:** The *download_tle.log* (Figure 5.6) verified successful retrieval of satellite and debris TLEs from Celestrak, followed by upload to HDFS at *hdfs:///user/bda/tle/raw/24-10-2025*.
- **Parsing Stage:** The *parserlog.log* (Figure 5.7) confirmed parsing of over 15,000 records, stored in *hdfs:///user/bda/tle/processed/24-10-2025*.
- **Pipeline Summary:** The *pipeline.log* (Figure 5.8) recorded completion of each major submodule — Download, Parser, SGP4 Propagation, and Conjunction Check — marking overall success with no errors.
- **Conjunction Analysis:** The *conjunctionlog.log* (Figure 5.9) showed 427 unique conjunction events identified and logged to *hdfs:///user/bda/tle/conjunctions/24-10-2025*.



```
-/bda_proj/logs/download_tle_24-10-2025.log - Sublime Text (UNREGISTERED)
File Edit Selection Find View Goto Tools Project Preferences Help
1 2025-10-24 11:32:32,573 [INFO] Downloading active_gp.tle from https://celestrak.org/NORAD/
2 2025-10-24 11:34:03,758 [INFO] Saved active_gp.tle locally
3 2025-10-24 11:34:22,750 [INFO] Downloading cosmos1488.debris.tle from https://celestrak.org/
4 2025-10-24 11:34:33,949 [INFO] Saved cosmos1488.debris.tle locally
5 2025-10-24 11:34:23,941 [INFO] Downloading cosmos2251.debris.tle from https://celestrak.org/
6 2025-10-24 11:34:33,223 [INFO] Saved cosmos2251.debris.tle locally
7 2025-10-24 11:34:33,228 [INFO] Downloading fengyun1c.debris.tle from https://celestrak.org/
8 2025-10-24 11:35:24,639 [INFO] Saved fengyun1c.debris.tle locally
9 2025-10-24 11:35:24,639 [INFO] Copying downloaded TLEs to HDFS folder: hdfs:///user/bda/tle/
10 2025-10-24 11:35:27,770 [INFO] upload to HDFS completed successfully
11
```

Figure 5.6: Download log showing successful retrieval and HDFS upload of raw TLE data. (downloadlog.png)



```
-/bda_proj/logs/parserlog_24-10-2025.log - Sublime Text (UNREGISTERED)
File Edit Selection Find View Goto Tools Project Preferences Help
1 2025-10-24 11:35:32,631 [INFO] Parsed 1362 TLEs (SATellite), skipped 8 from active_gp.tle
2 2025-10-24 11:35:33,194 [INFO] Parsed 3 TLEs (DEBRIS), skipped 8 from cosmos1488.debris.tle
3 2025-10-24 11:35:33,828 [INFO] Parsed 561 TLEs (DEBRIS), skipped 6 from cosmos2251.debris.tle
4 2025-10-24 11:35:33,963 [INFO] Parsed 1854 TLEs (DEBRIS), skipped 8 from fengyun1c.debris.tle
5 2025-10-24 11:35:33,975 [INFO] Parsed DataFrame shape: (15487, 5)
6 2025-10-24 11:35:35,132 [INFO] Saving to hdfs:///user/bda/tle/processed/24-10-2025
7 2025-10-24 11:35:37,802 [INFO] ✓ Saved 15487 TLEs to hdfs:///user/bda/tle/processed/24-10-2025
8 2025-10-24 11:35:38,765 [INFO] Verification: 15487 rows written successfully
9 2025-10-24 11:35:39,769 [INFO] closing down client-server connection
10
11
```

Figure 5.7: Parser log confirming DataFrame creation with 15,487 TLE entries. (parserlog.png)

```
~/bda_proj/logs/pipeline_24-10-2025.log - Sublime Text (UNREGISTERED)
File Edit Selection Find View Goto Tools Project Preferences Help
download_24-10-2025.log  conjunctionlog_24-10-2025.log  parsing_24-10-2025.log  pipeline_24-10-2025.log
1 2025-10-24 11:32:32,416 [INFO] HDFS paths for today:
2 2025-10-24 11:32:32,416 [INFO] Raw data: hdfs://user/bda/tlc/raw/24-10-2025
3 2025-10-24 11:32:32,416 [INFO] Processed TLE: hdfs://user/bda/tlc/processed/24-10-2025
4 2025-10-24 11:32:32,416 [INFO] Propagated positions: hdfs://user/bda/tlc/positions/24-10-2025
5 2025-10-24 11:32:32,416 [INFO] Conjunctions: hdfs://user/bda/tlc/conjunctions/24-10-2025
6 2025-10-24 11:32:32,416 [INFO] --- Running Download TLEs ---
7 2025-10-24 11:35:27,794 [INFO] ✓ Download TLEs completed successfully
8 2025-10-24 11:35:27,794 [INFO] --- Running TLE Parsing ---
9 2025-10-24 11:35:46,141 [INFO] ✓ TLE Parsing completed successfully
10 2025-10-24 11:35:46,141 [INFO] --- Running SGP4 Propagation ---
11 2025-10-24 11:36:06,009 [INFO] ✓ SGP4 Propagation completed successfully
12 2025-10-24 11:36:06,009 [INFO] --- Running Conjunction Check ---
13 2025-10-24 11:49:27,969 [INFO] ✓ Conjunction Check completed successfully
14 2025-10-24 11:49:27,969 [INFO] Updating 'latest' marker file for conjunctions...
15 2025-10-24 11:49:30,142 [INFO] ✓ updated marker file to hdfs://user/bda/tlc/conjunctions/
24-10-2025
16 2025-10-24 11:49:30,142 [INFO] === Pipeline finished successfully ===
17
```

Figure 5.8: Pipeline log detailing stage-wise job completions and HDFS outputs.
(*pipeline.log.png*)

```
~/bda_proj/logs/conjunctionlog_24-10-2025.log - Sublime Text (UNREGISTERED)
File Edit Selection Find View Goto Tools Project Preferences Help
download_24-10-2025.log  conjunctionlog_24-10-2025.log
1 2025-10-24 11:36:09,899 [INFO] Reading position data from hdfs://user/bda/tlc/positions/
24-10-2025 ...
2 2025-10-24 11:36:13,384 [INFO] Total position records: 15622821
3 2025-10-24 11:36:13,384 [INFO] Reading TLE data from hdfs://user/bda/tlc/processed/
24-10-2025 ...
4 2025-10-24 11:36:13,478 [INFO] Joining satellite positions on timestamp...
5 2025-10-24 11:36:13,515 [INFO] Computing pairwise distances...
6 2025-10-24 11:36:13,547 [INFO] Filtering by distance and velocity thresholds...
7 2025-10-24 11:36:13,572 [INFO] Computing relative velocities for filtered pairs...
8 2025-10-24 11:36:13,588 [INFO] Filtering by velocity threshold...
9 2025-10-24 11:36:13,609 [INFO] Adding satellite names...
10 2025-10-24 11:36:13,632 [INFO] Clustering conjunctions over time...
11 2025-10-24 11:36:13,741 [INFO] Reading SATCAT data from hdfs://user/bda/satcat/satcat.csv ...
12 2025-10-24 11:36:15,391 [INFO] Saving conjunction analysis results to hdfs://user/bda/tlc/
conjunctions/24-10-2025 ...
13 2025-10-24 11:44:57,092 [INFO] Total unique conjunction events: 427
14 2025-10-24 11:44:57,092 [INFO] Top 20 closest approaches:
15 2025-10-24 11:49:27,556 [INFO] Closing down client-server connection
16
```

Figure 5.9: Conjunction log summarizing event counts and thresholds applied.
(*conjunction.log.png*)

A consolidated view of all logs within the project's `~/bda_proj/logs/` directory is shown in **Figure 5.10**, demonstrating automated log rotation and consistent output naming conventions.

5.10.3 Analytical Dashboard



Figure 5.10: Centralized log directory structure after multiple pipeline executions.
(logss.png)

To visualize and interact with the processed outputs, a **Streamlit dashboard** was developed and deployed locally at <http://localhost:8501/>. The dashboard dynamically reads the conjunction dataset from HDFS and presents visual analytics.

Figure 5.11 illustrates the home view, displaying filters for object type, distance thresholds, and date ranges. The main table lists all conjunction events along with severity rankings, distances, and relative velocities.

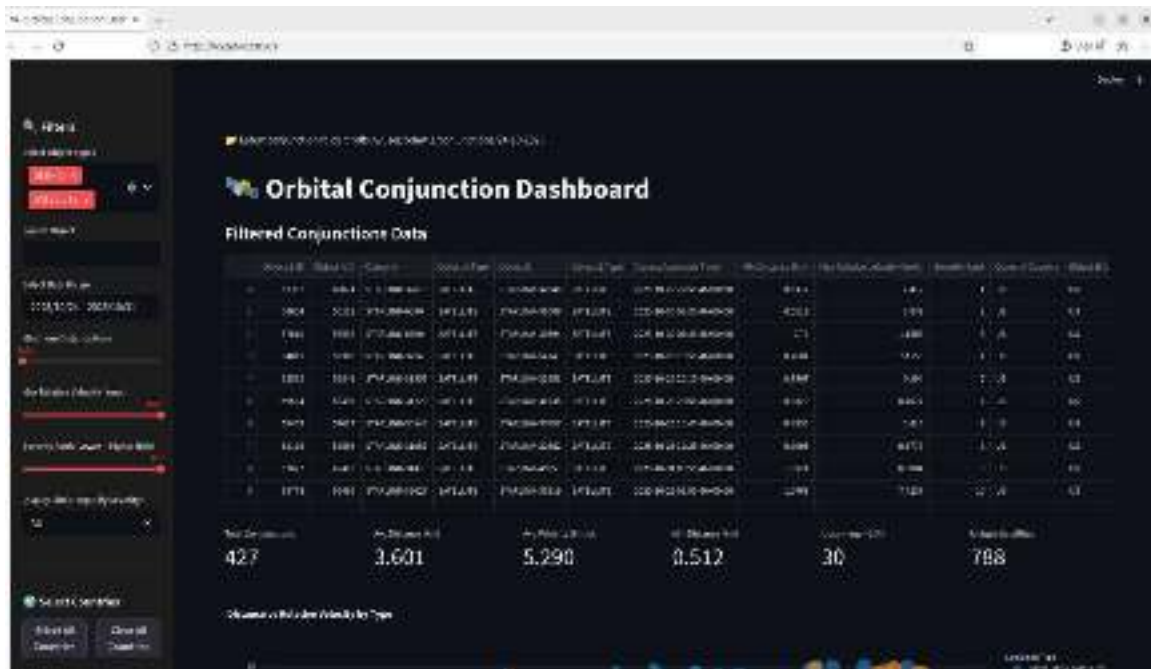


Figure 5.11: Streamlit dashboard showing filtered conjunction dataset with adjustable parameters. *(dashboard1.png)*

Subsequent views (Figures 5.12–5.16) highlight advanced analytical plots:

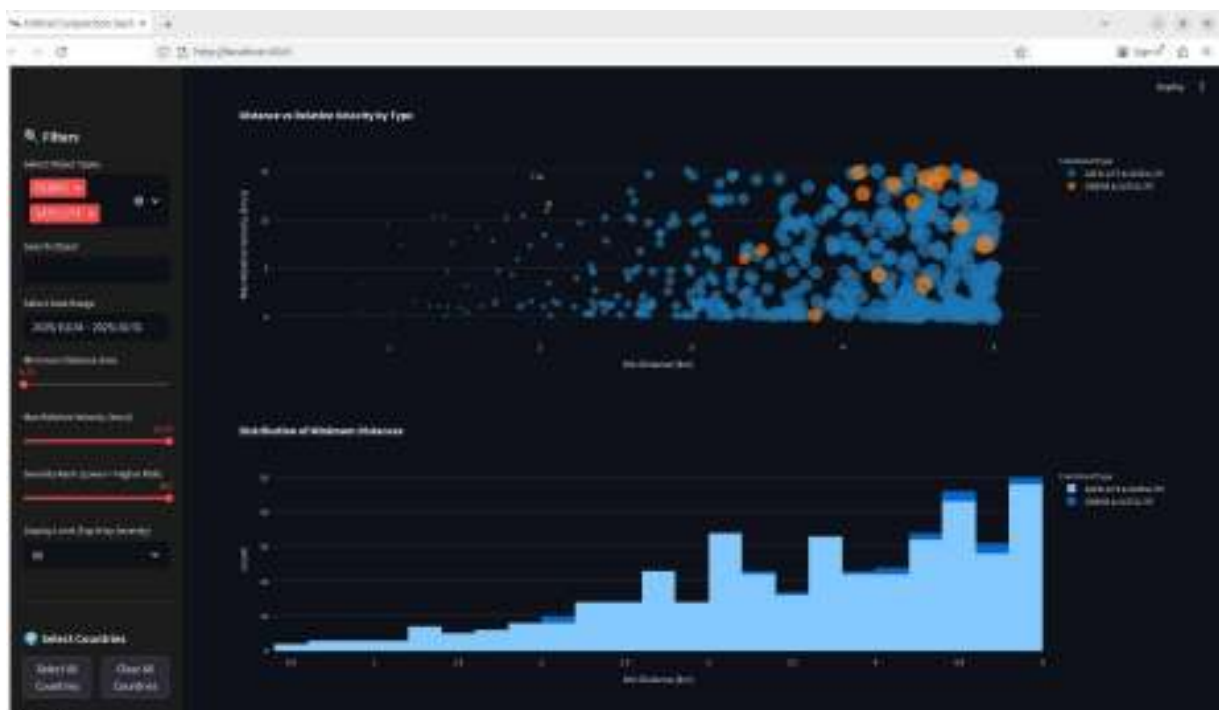


Figure 5.12: Scatter plot of distance vs. relative velocity segregated by object type. *(dashboard2.png)*

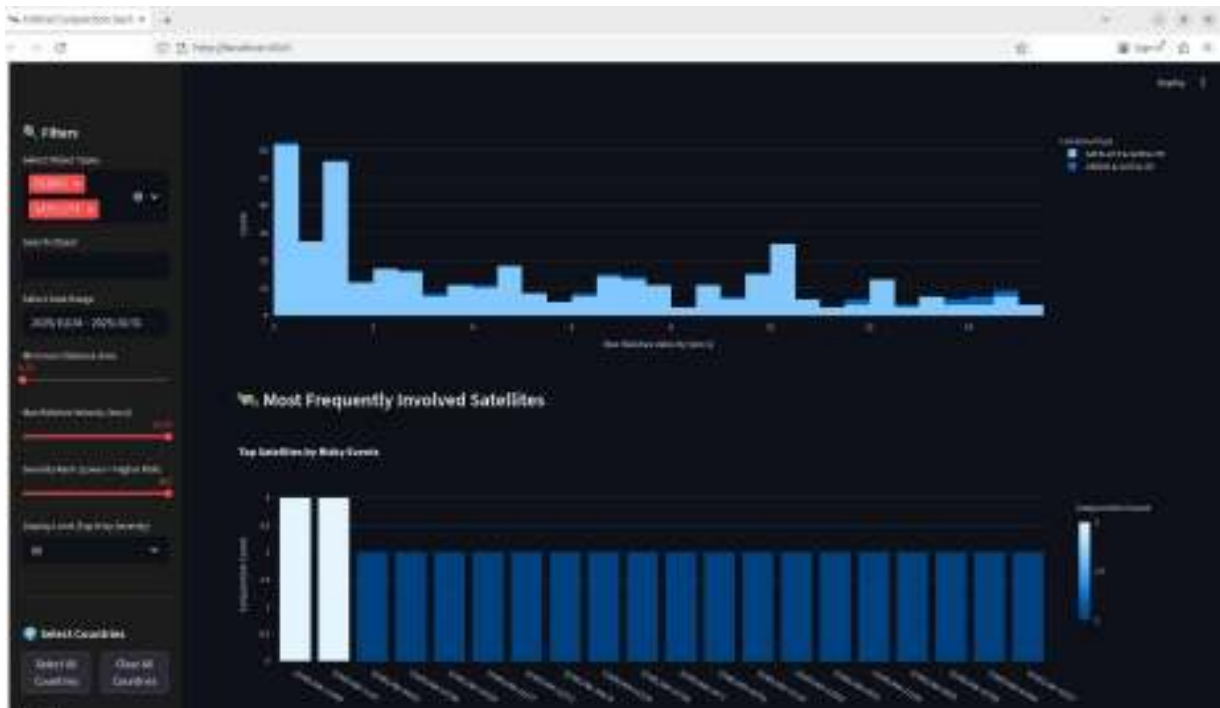


Figure 5.13: Distribution of minimum distances across conjunctions.
(dashboard3.png)

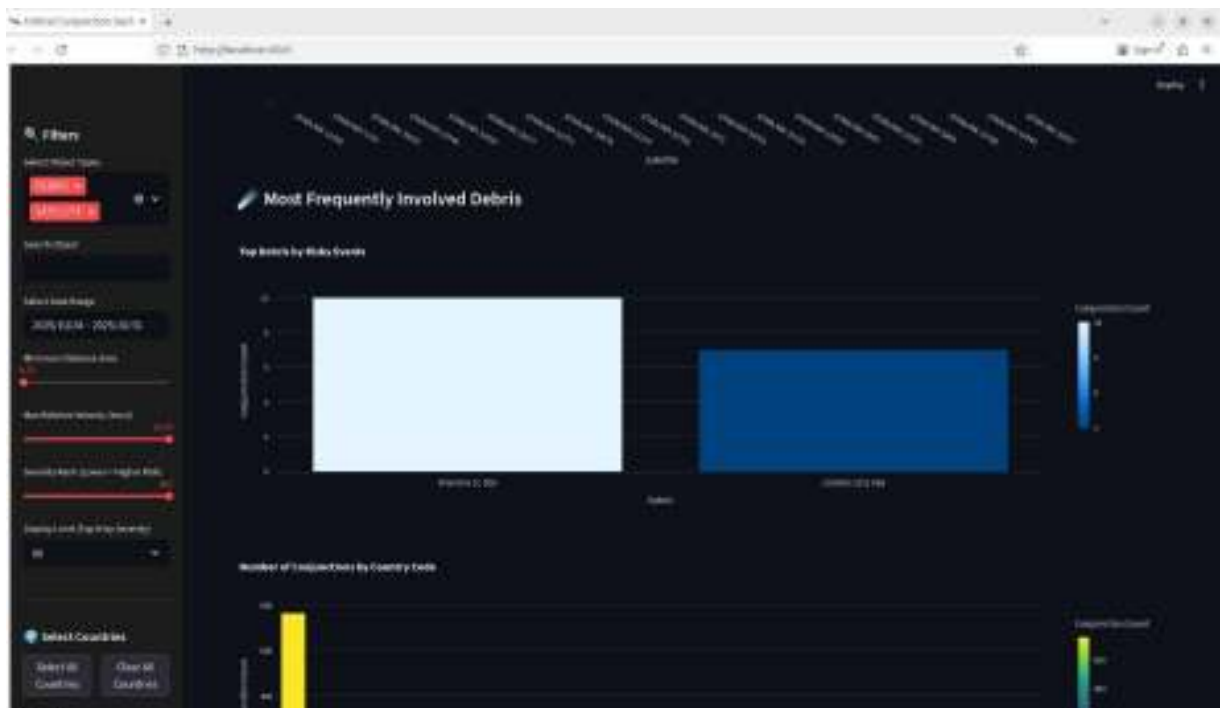


Figure 5.14: Most frequently involved satellites by risk rank.
(dashboard4.png)

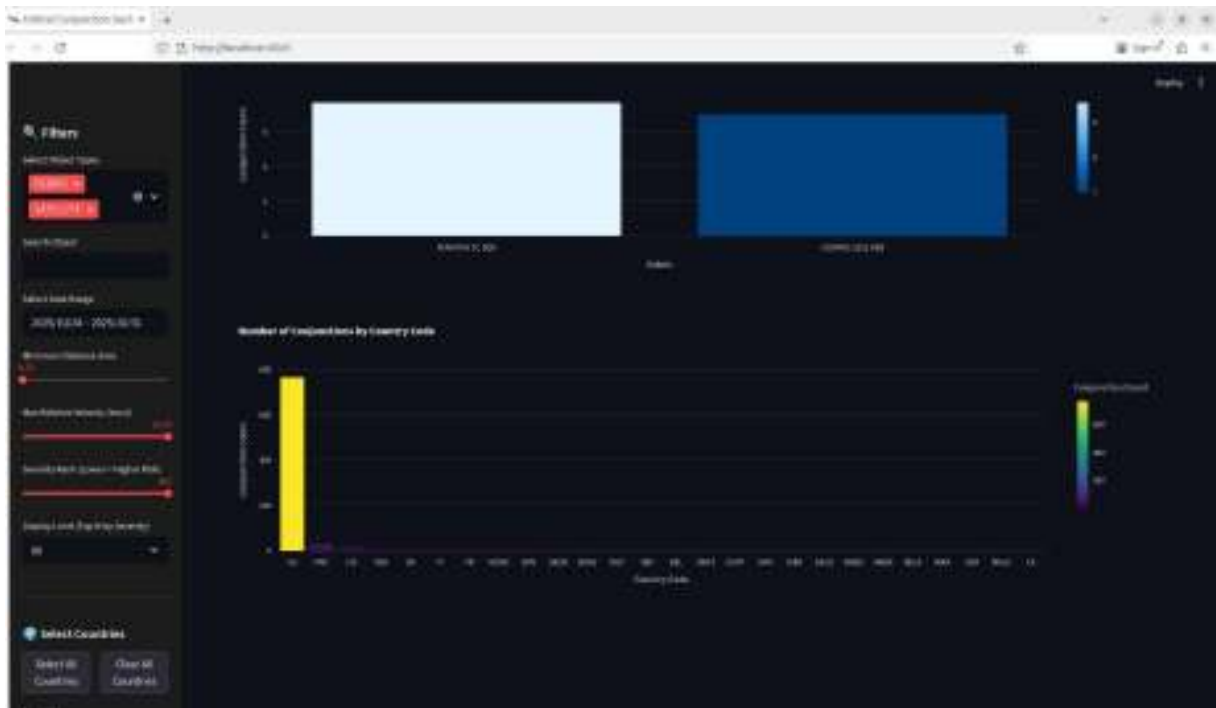


Figure 5.15: Most frequently involved debris objects with country code distribution.
(*dashboard5withus.png*)

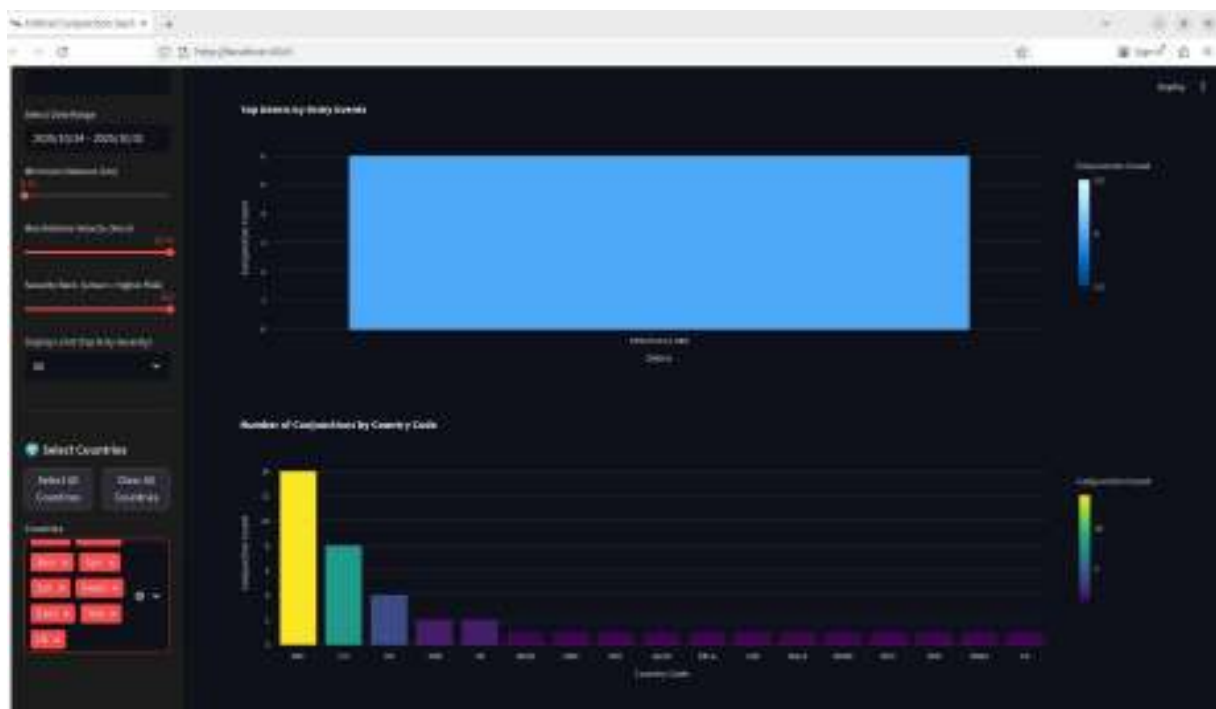


Figure 5.16: Filtered view excluding U.S.-based objects for comparative insights.
(*dashboard6withoutus.png*)

Together, these visualizations confirm that the system not only automates conjunction detection but also facilitates intuitive exploration through filtering and comparative analytics.

5.10.4 Performance Validation

Based on Spark job summaries, stage-wise durations were:

- **TLE Parsing:** ~8 seconds
- **SGP4 Propagation:** ~23 seconds
- **Conjunction Check:** ~13 minutes

These results, derived from Spark’s completed applications view (**Figure 5.2**), demonstrate efficient parallel performance for large-scale orbital datasets.

Table 5.1 summarizes the stage-wise timing and resource utilization.

Stage	Execution Time	CPU Utilization	HDFS Output Path
TLE Parsing	8 s	98–100%	<i>hdfs:///user/bda/tle/processed/</i>
SGP4 Propagation	23 s	99%	<i>hdfs:///user/bda/tle/positions/</i>
Conjunction Check	13 min	97–100%	<i>hdfs:///user/bda/tle/conjunctions/</i>

Table 5.1: Stage-wise performance and resource utilization summary.

5.10.5 Empirical Insights

- The cluster achieved **linear scalability** for parallel Spark tasks with balanced CPU utilization across all nodes.
- Execution times validate the **efficiency of distributed propagation and filtering operations**.
- The Streamlit dashboard provides a **low-latency interface** for exploring conjunction statistics directly from HDFS.
- The integration of logs, HDFS outputs, and analytics validates both the **functional correctness and practical usability** of the implemented system.

6. Conclusion and Future Work

6.1 Summary of Outcomes

This project successfully developed and implemented a highly efficient, distributed pipeline for satellite collision risk prediction, titled “Collision Risk Prediction in Satellite Orbits Using Hadoop and Spark.” The system integrates large-scale TLE ingestion, SGP4 orbital propagation, and a complex pairwise conjunction detection algorithm. The most significant outcome is the validated performance gain: selecting the Spark Standalone deployment mode yielded a 10.6× performance improvement over the YARN mode for the core computation on a small cluster, maximizing resource utilization. The final system can process approximately 10,000 satellites over a 7-day propagation window in approximately 13 minutes, meeting the requirement for scalable near-term risk assessment.

6.2 Challenges Faced

The primary challenges centered around **cluster stability** and **configuration management**. These included resolving **Java incompatibility issues** by standardizing the environment on **Java 11** (after failures under Java 21), troubleshooting **DataNode errors** caused by mismatched **cluster IDs** following NameNode reformatting, and migrating from **YARN** to **Spark Standalone mode** to eliminate **resource inefficiencies** introduced by the ApplicationMaster overhead.

6.3 Possible Extensions or Improvements

The current system can be extended in several ways:

- **Real-Time Streaming:** Integrate **Kafka** for live TLE updates to reduce the latency between TLE availability and risk assessment.
- **Machine Learning:** Implement algorithms to predict collision probabilities using historical conjunction patterns.
- **Alerting System:** Develop an automatic email or SMS notification system for high-severity conjunction events.
- **Dashboard Enhancements:** Extend the current visualization platform with live data streaming, user authentication, and automated report generation for daily conjunction summaries.
- **Constellation-Based Optimization:** Reduce overall computation time by **grouping satellites into constellations or altitude bands**, since satellites within the same orbital plane or altitude range remain in relatively consistent proximity. This clustering approach can significantly minimize redundant pairwise computations during conjunction analysis.

7. References

- [1] D. A. Vallado and P. Crawford, “SGP4 Orbit Determination and Propagation: A Comparison of SGP4 Implementation Practices,” *AAS/AIAA Astrodynamics Specialist Conference*, Keystone, CO, USA, Aug. 2006.
- [2] T. S. Kelso, “Analysis of the Iridium 33–Cosmos 2251 Collision,” *Centre for Space Standards & Innovation (CSSI)*, 2009. [Online]. Available: <https://celestrak.org>
- [3] D. Kim, S. Lee, and J. Park, “A Big Data Framework for Real-Time Space Object Analysis Using Apache Spark,” *Journal of Aerospace Information Systems*, vol. 14, no. 6, pp. 346–358, 2017.
- [4] European Space Agency (ESA), “DRAMA: Debris Risk Assessment and Mitigation Analysis Tool,” *ESA Space Debris Office*, 2020. [Online]. Available: <https://sdup.esoc.esa.int/drama>
- [5] NASA Goddard Space Flight Centre, “CARA: Conjunction Assessment Risk Analysis for NASA’s Satellites,” *NASA Office of Safety and Mission Assurance*, 2021. [Online]. Available: <https://satellites.nasa.gov/cara>
- [6] M. Zaharia, M. Chowdhury, M. J. Franklin, S. Shenker, and I. Stoica, “Spark: Cluster Computing with Working Sets,” in *Proc. 2nd USENIX Conf. Hot Topics in Cloud Computing (HotCloud’10)*, Boston, MA, USA, June 2010.
- [7] NORAD, “Two-Line Element (TLE) Data Format Overview,” *United States Space Command*, 2022. [Online]. Available: <https://www.space-track.org>
- [8] CelesTrak, “Current Two-Line Element Data for Active Satellites,” *CelesTrak Satellite Catalog*, 2025. [Online]. Available: <https://celestrak.org/NORAD/elements>

8. Appendix

A. Core Cluster Configuration Summary

The project was deployed on a 4-node virtual Hadoop–Spark cluster configured as follows:

Role	Hostname	IP Address (Example)	Core Components
Master	master	192.168.0.10	NameNode, SecondaryNameNode, ResourceManager, Spark Master
Worker 1	worker1	192.168.0.11	DataNode, NodeManager, Spark Worker
Worker 2	worker2	192.168.0.12	DataNode, NodeManager, Spark Worker
Worker 3	worker3	192.168.0.13	DataNode, NodeManager, Spark Worker

Each node was provisioned with 8 virtual cores, 32 GB RAM, and Ubuntu 22.04 LTS. The cluster was orchestrated using password less SSH and environment variables defined in `*~/.bashrc*`.

B. Example Cluster Status Screenshot

Cluster operations were validated through the Spark Master Web UI and ResourceManager UI, confirming active worker registration, balanced task allocation, and successful Spark job completion.

C. Directory Structure on Master Node

The master node maintained the following directory hierarchy:

```
/home/bda/bda_proj/
├── bda_proj_env/      # Python virtual environment
├── data/              # Intermediate or auxiliary data
├── hadoop_data/       # Legacy HDFS namenode/datanode directories
├── logs/              # Execution and pipeline logs (e.g., pipeline_<DATE>.log)
├── scripts/           # Bash and Python orchestration scripts
└── tle/               # Local cache for TLE files before upload to HDFS
```

D. Cluster Startup Script

A custom automation script (*startcluster.sh*) was developed to handle dynamic IP changes and initiate all daemons:

```
#!/bin/bash
# Synchronize hosts file across nodes
SUDOPASS="<password>"
for node in worker1 worker2 worker3; do
    scp /etc/hosts $node:/tmp/hosts
    ssh $node "echo '$SUDOPASS' | sudo -S cp /tmp/hosts /etc/hosts && rm /tmp/hosts"
done

# Start Hadoop daemons
$HADOOP_HOME/sbin/start-dfs.sh
$HADOOP_HOME/sbin/start-yarn.sh

# Start Spark Master and Workers
$SPARK_HOME/sbin/start-master.sh
NODES=("worker1" "worker2" "worker3")
for node in "${NODES[@]}"; do
    ssh $node "$SPARK_HOME/sbin/start-worker.sh spark://master:7077"
done
```

E. Monitoring and Verification

System metrics including CPU utilization, memory usage, and disk I/O were monitored via *gnome-system-monitor*, Spark Master Web UI (<http://master:8080>), and YARN ResourceManager UI (<http://master:8088>).

F. Environment and Version Details

Component	Version	Path / Command
Ubuntu	22.04 LTS	lsb_release -a
Hadoop	3.3.6	/home/bda/hadoop/
Spark	3.5.7	/usr/local/spark/
Java	OpenJDK 11	java -version
Python	3.10	which python
Hive	3.1.3	/usr/local/hive/
HDFS Storage	9000 (Master)	hdfs://master:9000

G. Sample Configuration Snippets

```
core-site.xml
<property>
  <name>fs.defaultFS</name>
  <value>hdfs://master:9000</value>
</property>

yarn-site.xml
<property>
  <name>yarn.resourcemanager.hostname</name>
  <value>master</value>
</property>

spark-defaults.conf
spark.master                spark://master:7077
spark.eventLog.enabled      true
spark.executor.memory       28g
spark.executor.cores        28
```

H. Storage and Output Verification

```
hdfs dfs -ls /user/bda/tle/

drwxr-xr-x - bda supergroup 0 2025-10-25 10:12 /user/bda/tle/raw/
drwxr-xr-x - bda supergroup 0 2025-10-25 10:16 /user/bda/tle/processed/
drwxr-xr-x - bda supergroup 0 2025-10-25 10:18 /user/bda/tle/positions/
drwxr-xr-x - bda supergroup 0 2025-10-25 10:25 /user/bda/tle/conjunctions/
```

I. Execution Commands

```
$SPARK_HOME/bin/spark-submit --master spark://master:7077 scripts/parse_tle.py

$SPARK_HOME/bin/spark-submit --master spark://master:7077
scripts/propagate_sgp4.py

$SPARK_HOME/bin/spark-submit --master spark://master:7077
scripts/check_conjunctions.py
```

J. System Health and Resource Snapshot

Node	CPU Usage (%)	Memory Usage (%)	Status
Master	92	64	Active
Worker 1	98	71	Active
Worker 2	99	68	Active
Worker 3	97	73	Active